

Foundations of 4D/6D Object Capture for Virtual Environments

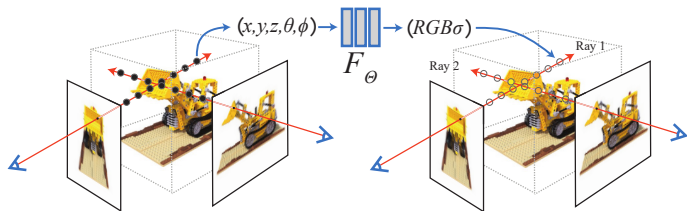
Gaussian Splatting



March 10, 2026



→ **Neural Radiance Fields**



Today: Gaussian Splatting

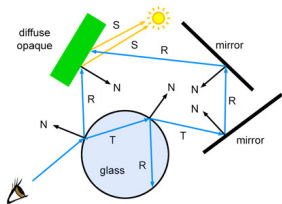
Very detailed results and realistic novel view synthesis



Problem: Although the visual quality of NeRF is high, rendering takes ~ 30 seconds per frame

Volume Rendering Surface Rendering

Surface Rendering



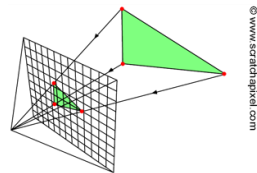
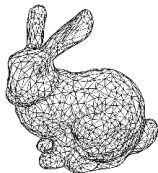
© www.scratchapixel.com

- Compute relevant surface point
- Evaluate BRDF at surface point

→ Empty space in scene completely skipped

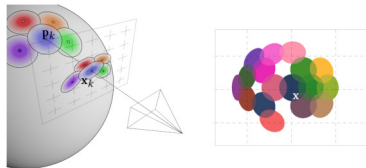
Reminder: Meshes can be very efficiently rendered via Rasterization

- ▶ Vertices
- ▶ Faces



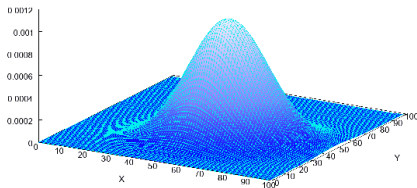
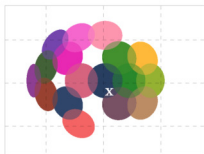
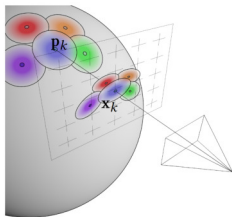
Question: Can we apply a similar technique to unstructured data like points/surfels?

- ▶ Positions
- ▶ (Normals)
- ▶ (Radii)



→ (Volume) Splatting

Idea: Use elliptical Gaussian kernels for splatting → Elliptical Weighted Average (EWA) Volume Splatting¹



$$\mathcal{N}^{3D}(\mathbf{x} | \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{|2\pi\boldsymbol{\Sigma}|^{\frac{1}{2}}} e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})}$$

Important Property: Gaussians are closed under specific operations

- ▶ Affine transformations
- ▶ Integration along coordinate axis

¹M. Zwicker et al. "EWA Volume Splatting". In: Visualization. IEEE. 2001.

Important Property: Gaussians are closed under specific operations

- **Affine transformations:** Given an affine transformation

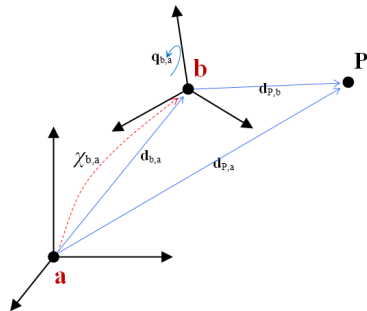
$$T = \left[\begin{array}{c|c} A & b \\ \hline 0 & 1 \end{array} \right]$$

the Gaussian w.r.t. the transformed variable is

$$y = Ax + b$$

$\Downarrow$

$$\mathcal{N}^{3D}(\underbrace{T^{-1}y}_x \mid \mu, \Sigma) = \frac{1}{|A^{-1}|} \mathcal{N}^{3D}(y \mid A\mu + b, A\Sigma A^T)$$



→ We can, for instance, "switch" Gaussians between coordinate system and much more

Important Property: Gaussians are closed under specific operations

- **Integration along coordinate axis**, e.g. z axis:

$$\int_{\mathbb{R}} \mathcal{N}^{3D}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) d x_3 = \mathcal{N}^{2D}(\mathbf{x}^{2D} \mid \boldsymbol{\mu}^{2D}, \boldsymbol{\Sigma}^{2D})$$

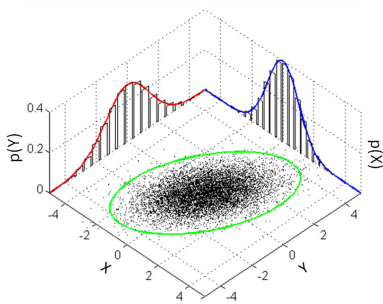
where the respective dimension is discarded:

$$\mathbf{x}^{2D} = (x_1, x_2)^T$$

$$\boldsymbol{\mu}^{2D} = (\mu_1, \mu_2)^T$$

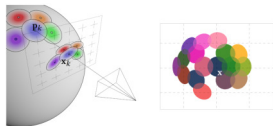
$$\boldsymbol{\Sigma}^{2D} = \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix}$$

→ We can "reduce" a 3D Gaussian to a 2D Gaussian



Task: Project 3D Gaussian $\mathcal{N}^{3D}$ into 2D "image-space" Gaussian $\mathcal{N}^{2D}$

1. Transform from world coordinates to camera coordinates
2. Perform perspective projection



Step 1: Every rigid transformations is also an affine transformation

$$T = \left[\begin{array}{c|c} R & t \\ \hline \mathbf{0} & 1 \end{array} \right]$$

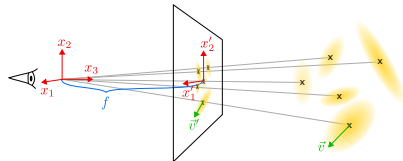
→ Apply **Affine Transformation property** of Gaussians:

$$\mathbf{x}_{\text{cam}} = \mathbf{R} \mathbf{x}_{\text{world}} + \mathbf{t} \quad \Rightarrow \quad \mathcal{N}^{3D}(\underbrace{T^{-1} \mathbf{x}_{\text{cam}}}_{\mathbf{x}_{\text{world}}} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{|\mathbf{R}^{-1}|} \mathcal{N}^{3D}(\mathbf{x}_{\text{cam}} \mid \underbrace{\mathbf{R} \boldsymbol{\mu} + \mathbf{t}}_{\boldsymbol{\mu}_{\text{cam}}}, \underbrace{\mathbf{R} \boldsymbol{\Sigma} \mathbf{R}^T}_{\boldsymbol{\Sigma}_{\text{cam}}})$$

Step 2: Consider the formula for perspective projection

$$(p_x, p_y)^T = \pi(\mathbf{K} \cdot (v_x, v_y, v_z)^T) = \underbrace{\left(\frac{f_x v_x}{v_z} + c_x, \frac{f_y v_y}{v_z} + c_y \right)^T}_{\Pi(\mathbf{v})}$$

→ Projection Π is not affine, due to perspective division

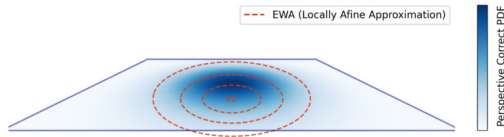


Solution: Linearize Π around Gaussian center μ_{cam}

$$\Pi(\mathbf{v}) \approx \Pi(\mu_{\text{cam}}) + \mathbf{J} \cdot (\mathbf{v} - \mu_{\text{cam}})$$

→ Taylor expansion

$$\mathbf{J} = \frac{\partial \Pi}{\partial \mathbf{v}}(\mu_{\text{cam}}) = \begin{pmatrix} \frac{f_x}{\mu_z} & 0 & -\frac{f_x \mu_x}{\mu_z^2} \\ 0 & \frac{f_y}{\mu_z} & -\frac{f_y \mu_y}{\mu_z^2} \end{pmatrix} \in \mathbb{R}^{2 \times 3}$$



Step 2: Consider the (linearized) formula for perspective projection

$$\Pi(\mathbf{x}) \approx \Pi(\boldsymbol{\mu}_{\text{cam}}) + \mathbf{J} \cdot (\mathbf{x} - \boldsymbol{\mu}_{\text{cam}})$$

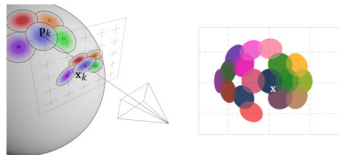
→ Apply **Affine Transformation property** of Gaussians:

$$\begin{aligned} \mathcal{N}^{3\text{D}}(\underbrace{\Pi^{-1}(\mathbf{x}_{\text{pixel}})}_{\mathbf{x}_{\text{cam}}} \mid \boldsymbol{\mu}_{\text{cam}}, \boldsymbol{\Sigma}_{\text{cam}}) &\approx \frac{1}{|\mathbf{J}^{-1}|} \mathcal{N}^{2\text{D}}(\mathbf{x}_{\text{pixel}} \mid \Pi(\boldsymbol{\mu}_{\text{cam}}), \mathbf{J} \boldsymbol{\Sigma}_{\text{cam}} \mathbf{J}^T) \\ &= \frac{1}{|\mathbf{R}^{-1}| |\mathbf{J}^{-1}|} \mathcal{N}^{2\text{D}}(\mathbf{x}_{\text{pixel}} \mid \Pi(\mathbf{R} \boldsymbol{\mu} + \mathbf{t}), \mathbf{J} \mathbf{R} \boldsymbol{\Sigma} \mathbf{R}^T \mathbf{J}^T) \end{aligned}$$

- ▶ 2D Mean: $\boldsymbol{\mu}^{2\text{D}} = \Pi(\mathbf{R} \boldsymbol{\mu} + \mathbf{t}) \in \mathbb{R}^2$
- ▶ 2D Covariance $\boldsymbol{\Sigma}^{2\text{D}} = \mathbf{J} \mathbf{R} \boldsymbol{\Sigma} \mathbf{R}^T \mathbf{J}^T \in \mathbb{R}^{2 \times 2}$

Note: Formally, $|\mathbf{J}^{-1}|$ does not exist since $\mathbf{J}$ is non-square (2×3)!

→ Not an issue: With **Integration along z axis** "almost" the same



Question: How can we blend multiple Gaussians together? → **Volumetric Rendering**

$$\hat{C} = \int_{t_n}^{t_f} T(t_n \rightarrow s) \sigma(s) \mathbf{c}(s) ds + T(t_n \rightarrow t_f) \hat{C}_{\text{bg}}$$

$$T(a \rightarrow b) = e^{-\int_a^b \sigma(u) du}$$

Idea: Reformulate this equation with these assumptions:

- Density $\sigma(t)$ is **weighted combination** of reconstruction kernels $r_i(t)$

$$\sigma(t) = \sum_{i=1}^N w_i r_i(t)$$

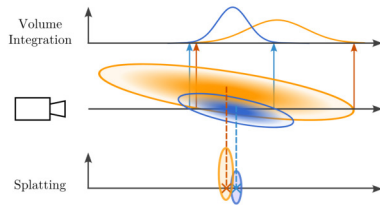
- **No overlapping** between reconstruction kernels
- Reconstruction kernels are **sorted front-to-back**

- **No self-occlusion** within reconstruction kernels
- **Linearized Transmittance** via Taylor expansion

$$e^{-x} \approx 1 - x$$

- **Constant color** within reconstruction kernels

$$\mathbf{c}(t) = \mathbf{c}_i \quad \forall t \in [t_i, t_{i+1})$$



With these assumptions, we can reformulate this equation (analogous for background term):

$$\begin{aligned}
 & \int_{t_n}^{t_f} T(t_n \rightarrow s) \sigma(s) c(s) ds \\
 &= \int_{t_n}^{t_f} c(s) \left(\sum_{i=1}^N w_i r_i(s) \right) e^{-\int_{t_n}^s \sum_{j=1}^N w_j r_j(u) du} ds && \left[\begin{array}{l} \text{Weighted combination} \\ + \text{Re-order terms} \end{array} \right] \\
 &= \sum_{i=1}^N w_i \left(\int_{t_i}^{t_{i+1}} c(s) r_i(s) ds \right) \prod_{j=1}^{i-1} e^{-w_j \int_{t_j}^{t_{j+1}} r_j(u) du} && \left[\begin{array}{l} \text{No overlapping} \\ + \text{Sorted front-to-back} \\ + \text{No self-occlusion} \end{array} \right] \\
 &= \sum_{i=1}^N c_i w_i \left(\int_{t_i}^{t_{i+1}} r_i(s) ds \right) \prod_{j=1}^{i-1} e^{-w_j \int_{t_j}^{t_{j+1}} r_j(u) du} && [\text{Constant color}] \\
 &= \sum_{i=1}^N c_i \underbrace{w_i \left(\int_{t_i}^{t_{i+1}} r_i(s) ds \right)}_{\alpha_i} \prod_{j=1}^{i-1} \left(1 - \underbrace{w_j \left(\int_{t_j}^{t_{j+1}} r_j(u) du \right)}_{\alpha_j} \right) && [\text{Linearized Transmittance}]
 \end{aligned}$$

Thus, Volume Splatting is performed via the α -blending formula:

$$\hat{C} = \sum_{i=1}^N c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) + \hat{C}_{bg} \prod_{j=1}^N (1 - \alpha_j)$$

For the opacity α_i , use Gaussian kernels $\mathcal{N}^{3D}(\mathbf{x} | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$:

$$\begin{aligned} \alpha_i &= o_i \cdot G^{2D}(\mathbf{x}_{\text{pixel}} | \boldsymbol{\mu}_i^{2D}, \boldsymbol{\Sigma}_i^{2D}) \\ &= o_i \cdot e^{-\frac{1}{2}(\mathbf{x}_{\text{pixel}} - \boldsymbol{\mu}_i^{2D})^T (\boldsymbol{\Sigma}_i^{2D})^{-1} (\mathbf{x}_{\text{pixel}} - \boldsymbol{\mu}_i^{2D})} \end{aligned}$$



(a) Viewing Scenario

EWA Splatting

Observation: Constant factors
(normalization and determinants)
moved into **peak opacity** o_i



$$o_i \in \mathbb{R}_{\geq 0}$$

Density-Centric: Original w_i constant
→ Peak opacity is view-dependent

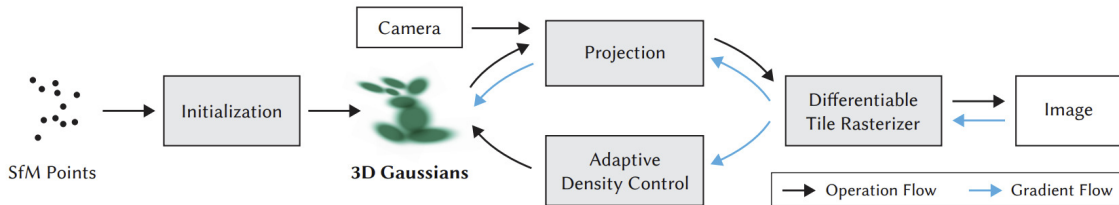
3D Gaussian Splatting (3DGS)



$$o_i \in [0, 1]$$

Opacity-Centric:
→ **Peak opacity is constant**

Based on this Volume Splatting technique, 3D Gaussian Splatting (3DGS)² performs reconstruction



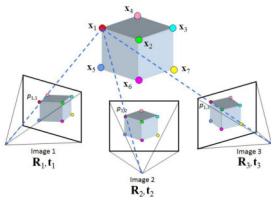
Differences to NeRF:

- ▶ 3D Gaussians vs. MLP F_{θ}
- ▶ Rasterization vs. Ray Marching

²B. Kerbl et al. "3D Gaussian Splatting for Real-Time Radiance Field Rendering". In: ACM Transactions on Graphics (TOG) 42.4 (2023).

In contrast to NeRF, 3DGS can be "warm-started" with coarse initialization

- Means μ : Use sparse 3D point cloud of matched correspondences from COLMAP



→ Essentially for free

- Covariances Σ : Choose isotropic shape

$$\Sigma = \bar{d}_3 \cdot \mathbf{I}$$

where $\bar{d}_3$ is mean distance to 3 closest points



The distribution of Gaussians can be suboptimal → Apply Densification and Pruning

Densification

- Under-reconstruction: Regions with not enough Gaussians to cover geometric details

$$\mu_{\text{cloned}} = \mu, \quad \text{momentum}(\nabla \mu_{\text{cloned}}) = \mathbf{0}$$

$$\Sigma_{\text{cloned}} = \Sigma$$

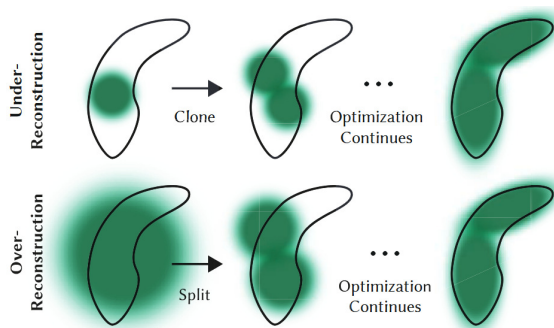
→ Clone, but use reset optimizer state

- Over-reconstruction: Regions where Gaussians cover (too) large area

$$\mu_{\text{split}} \sim \mathcal{N}(\cdot | \mu, \Sigma)$$

$$\Sigma_{\text{split}} = \frac{1}{\phi} \Sigma, \quad \phi = 1.6$$

→ Split into 2 smaller ones



Question: At which point should we perform Apply Densification and Pruning?

► **Densification:**

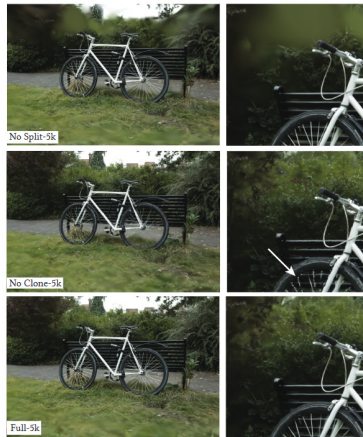
Perform cloning and splitting on Gaussians with large viewspace gradients

$$\left\| \frac{\partial \mathcal{L}}{\partial \mu^{2D}} \right\| > \tau_{\text{pos}} \quad , \quad \tau_{\text{pos}} = 0.0002$$

► **Pruning:**

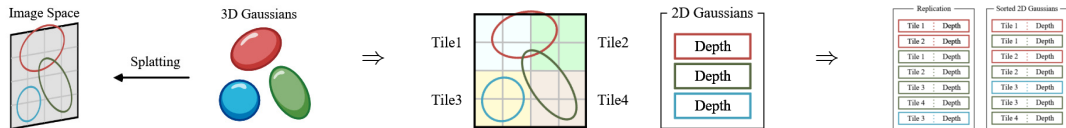
In addition, also remove Gaussians with low opacity

$$o < \epsilon_o \quad , \quad \epsilon_o = 0.005$$



Goal: Fast rasterization of anisotropic Gaussians $\rightarrow$ Real-time performance

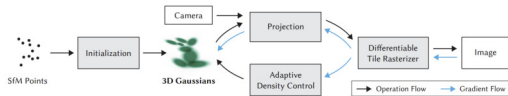
Idea: Truncate Gaussians at 99.7% confidence interval (i.e. 3 standard deviations)



1. Project Gaussians into G^{2D} in image space
2. Divide pixel grid into 16×16 tiles
3. Replicate 2D Gaussians falling into multiple tiles
4. Within each tile, sort according to their depth
5. Render via α -blending in sorted order:

$$\hat{C} = \sum_{i=1}^N c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) + \hat{C}_{bg} \prod_{j=1}^N (1 - \alpha_j)$$

Back to the pipeline, we can now consider the **Backward Pass**



Parameters to optimize:

$$\Theta = (\dots, (\mu_i, \Sigma_i, o_i, c_i(\mathbf{d})), \dots)^T$$

- ▶ Means $\mu_i \in \mathbb{R}^3$
- ▶ Covariances $\Sigma_i \in \mathbb{R}^{3 \times 3}$
- ▶ Peak opacities $o_i \in [0, 1]$
- ▶ View-dependent colors $c_i(\mathbf{d}) \in \mathbb{R}_{\geq 0}^3$



Final Rendering



3D Gaussian Visualization

Question: How to model these parameters Θ ?

- Covariances $\Sigma_i \in \mathbb{R}^{3 \times 3} \rightarrow$ Decompose into rotation and scaling

$$\Sigma = R S S^T R^T \quad \Rightarrow \quad \begin{aligned} R &= R_{\text{quaternion}}(q) \quad , \quad q \in \mathbb{H} \\ S &= \text{diag}(s) \quad , \quad s \in \mathbb{R}^3 \end{aligned}$$



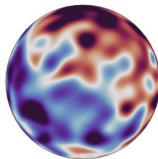
Ground Truth

Anisotropic

Isotropic

- View-dependent colors $c_i(d) \in \mathbb{R}_{\geq 0}^3 \rightarrow$ Spherical Harmonics

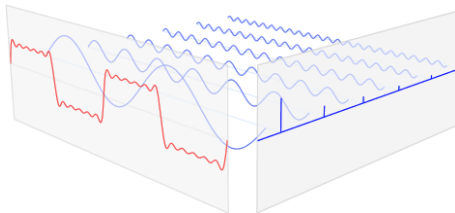
Question: How can we represent a function parameterized by directions?



Reminder: 1D functions (and similarly n -D functions) can be represented by frequencies:

Spatial domain

$$f(t) = \int_{\mathbb{R}} \hat{f}(\omega) e^{2\pi i \omega t} d\omega$$



Frequency domain

$$\hat{f}(\omega) = \int_{\mathbb{R}} f(t) e^{-2\pi i \omega t} dt$$

We can extend this concept to spheres:

Definition (Spherical Harmonics (SH))

On the unit sphere $\mathbb{S}^2$, the Spherical Harmonics are defined as

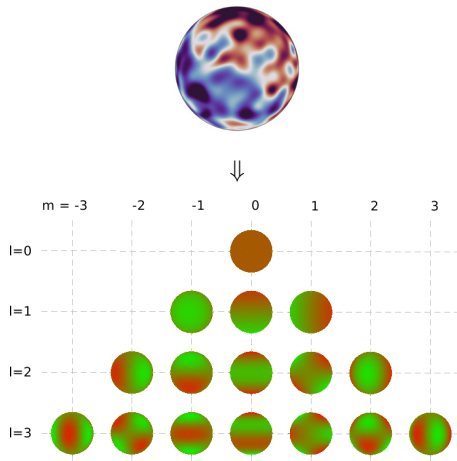
$$Y_l^m : [0, 2\pi] \times [0, \pi] \rightarrow \mathbb{C}$$

$$(\phi, \theta) \mapsto \sqrt{\frac{2l+1}{4\pi} \frac{(l-m)!}{(l+m)!}} P_l^m(\cos(\theta)) e^{im\phi}$$

where P_l^m denote the Associated Legendre Polynomials.

- Frequency bands denoted by l
- $2l + 1$ different functions per frequency band

$$\rightarrow \text{Use 3rd order SH: } c_i(\mathbf{d}) = \sum_{l=0}^3 \sum_m c_{l,m}^{\text{SH}} Y_l^m(\mathbf{d})$$



Backward Pass: Minimize combination of loss terms between input images C_k and rendered images $\hat{C}_k$
→ Pipeline fully differentiable

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_k (1 - \lambda) \cdot L_1(C_k, \hat{C}_k(\boldsymbol{\theta})) + \lambda \cdot \text{DSSIM}(C_k, \hat{C}_k(\boldsymbol{\theta}))$$

- ▶ Pixel-wise image loss similar to NeRF (L_1 here instead of L_2)

$$L_1(C_k, \hat{C}_k(\boldsymbol{\theta})) = \|C_k - \hat{C}_k(\boldsymbol{\theta})\|$$

- ▶ Additional dissimilarity loss

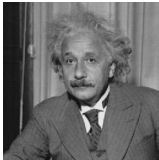
$$\text{DSSIM}(C_k, \hat{C}_k(\boldsymbol{\theta})) = \frac{1 - \text{SSIM}(C_k, \hat{C}_k(\boldsymbol{\theta}))}{2}$$

Question: How can we measure similarity between images other than via L_1 or L_2 distances?

SSIM: Structural Similarity Index Measure³

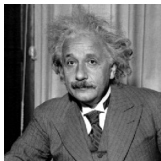
- ▶ Range: $[0, 1]$, higher is better
- ▶ Consider small window around pixels
- ▶ Compute statistics (mean + variance) of:
 - ▶ luminance
 - ▶ contrast
 - ▶ structure
- ▶ Perform weighted combination

Example:



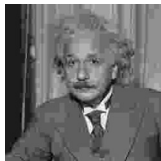
Ground Truth

Contrast Enhanced



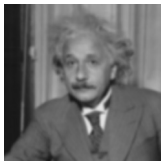
MSE = 144, SSIM = 0.9133

JPEG Artifacts



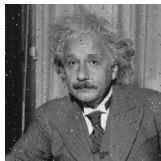
MSE = 142, SSIM = 0.6624

Blurred



MSE = 144, SSIM = 0.6940

Salt and Pepper Noise



MSE = 144, SSIM = 0.8317

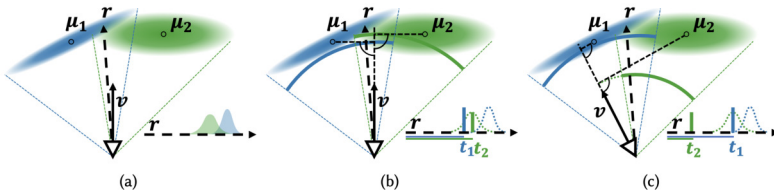
³Z. Wang et al. "Image Quality Assessment: From Error Visibility to Structural Similarity". In: IEEE Transactions on Image Processing 13.4 (2004).

Complex scenes can be optimized without compromises: Best quality + Best training time + Real-time rendering



Interactive Results: <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting>

Problem: Global sorting by depth of mean μ is highly sensitive to rotations of view direction⁴

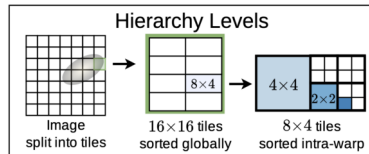


Idea: Use more granular sorting approach

1. Start with global searching of 16×16 tiles (same as 3DGS)
2. Do local resorting within each tile according to

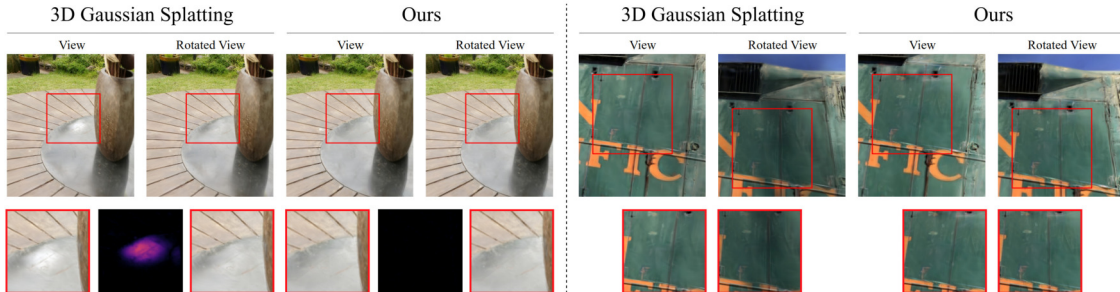
$$t_{\text{opt}} = \frac{d^T \Sigma^{-1} (\mu - o)}{d^T \Sigma^{-1} d}$$

→ Position along ray $r(t) = o + t d$ with maximum Gaussian probability



⁴L. Radl et al. "StopThePop: Sorted Gaussian Splatting for View-Consistent Real-time Rendering". In: ACM Transactions on Graphics (TOG) 43.4 (2024).

Results:

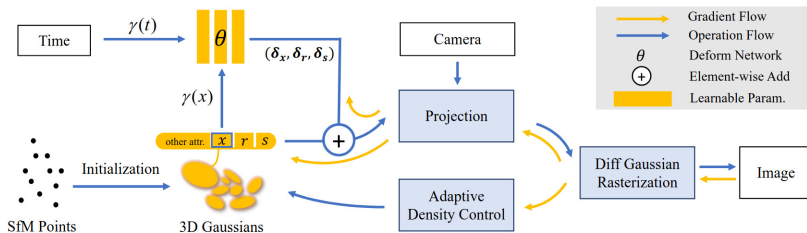


Interactive Results: <https://r4dl.github.io/StopThePop>

Problem: 3DGS can also only handle static scenes⁵

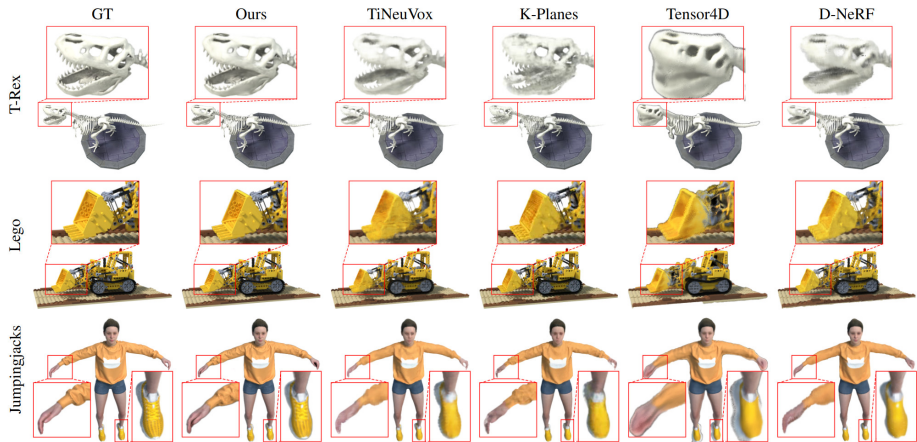
Idea:

- ▶ Train 1 network for the deformation + use 1 model for the reference static 3DGS ("canonical 3DGS")
- ▶ Warp Gaussians by deformation network via time-dependent offset $\Delta\mu, \Delta q, \Delta s$
- ▶ Perform Volume Splatting as usual



⁵Z. Yang et al. "Deformable 3D Gaussians for High-Fidelity Monocular Dynamic Scene Reconstruction". In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2024.

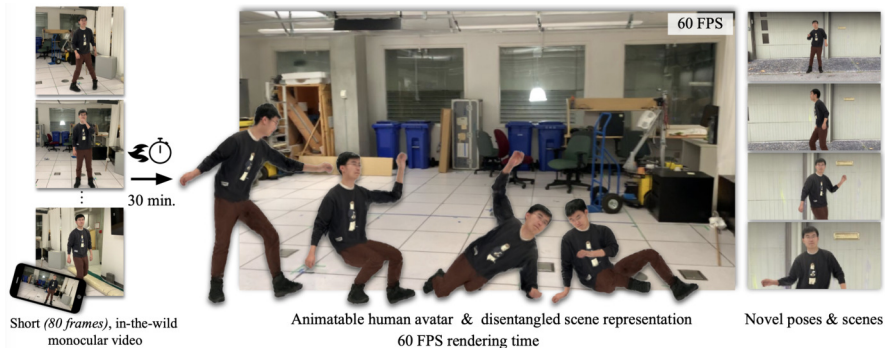
Results:



Interactive Results: <https://ingra14m.github.io/Deformable-Gaussians>

Extension: HUGS – Human Gaussian Splats

Task: Reconstructing an animatable human avatar from a monocular video⁶



Challenge: Separation of the dynamic human from its static background

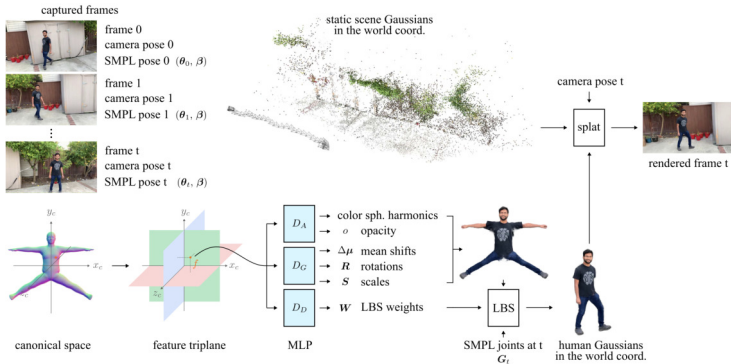
⁶M. Kocabas et al. "HUGS: Human Gaussian Splats". In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2024.

Extension: HUGS – Human Gaussian Splats

Idea: Use 3D Gaussian Splatting for the static scene and human, but combine it with a human template model

Dynamic 3D Gaussian Avatar:

- SMPL template model as a canonical space
- Attach Gaussian means to the template's surface
- Use a neural network to predict Gaussian parameters and LBS weights



Results:



Interactive Results: <https://machinelearning.apple.com/research/hugs>

High success in quality and render performance also lead to various applications

- Robotics

→ Teleoperation

- Medicine

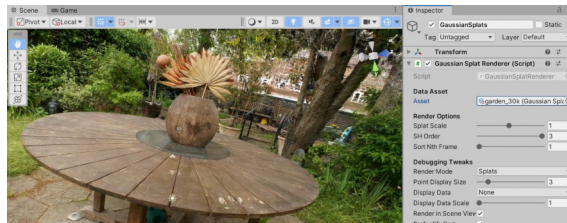
→ Endoscopic Scene Reconstruction

- Entertainment

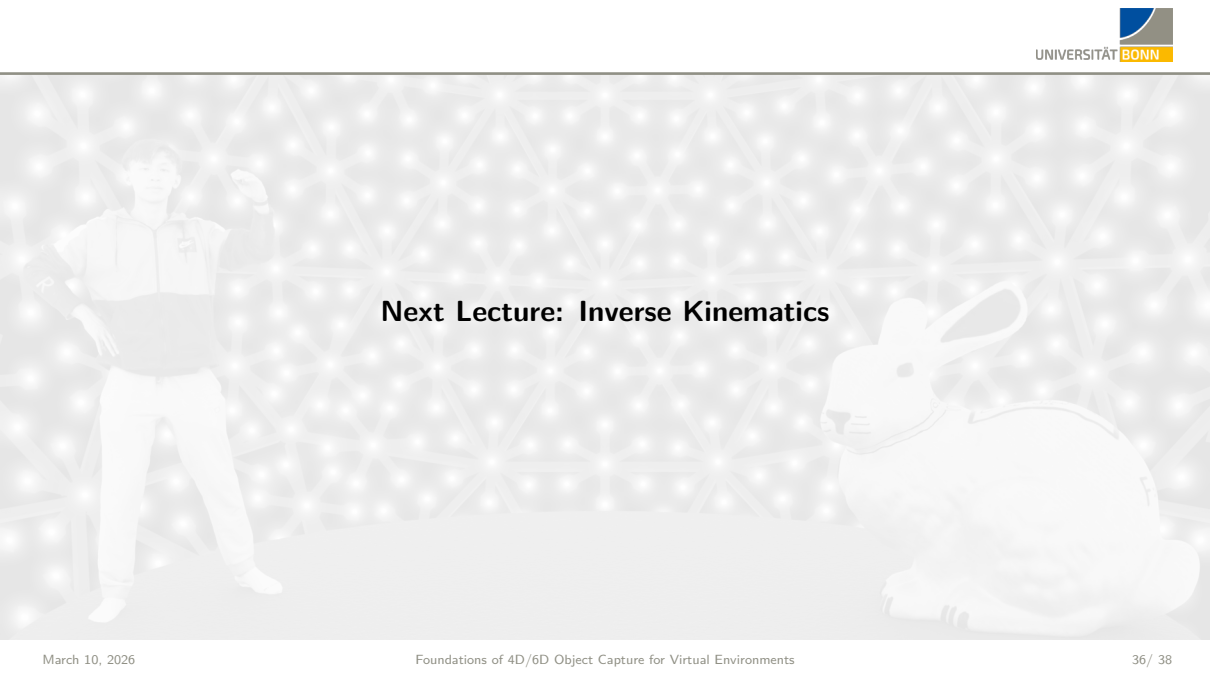
→ Interactive Web Viewer

<https://superspl.at/?sort=views&time=all>

- Integration into Unreal, Unity, Blender

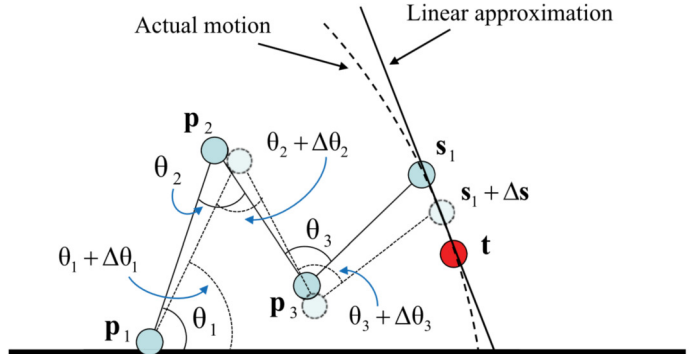


→ Becoming an industry standard
(glTF 2.0 Gaussian Splatting Extension)



Next Lecture: Inverse Kinematics

→ Inverse Kinematics



- [1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. “3D Gaussian Splatting for Real-Time Radiance Field Rendering”. In: ACM Transactions on Graphics (TOG) 42.4 (2023).
- [2] M. Kocabas, J.-H. R. Chang, J. Gabriel, O. Tuzel, and A. Ranjan. “HUGS: Human Gaussian Splats”. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2024.
- [3] L. Radl, M. Steiner, M. Parger, A. Weinrauch, B. Kerbl, and M. Steinberger. “StopThePop: Sorted Gaussian Splatting for View-Consistent Real-time Rendering”. In: ACM Transactions on Graphics (TOG) 43.4 (2024).
- [4] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli. “Image Quality Assessment: From Error Visibility to Structural Similarity”. In: IEEE Transactions on Image Processing 13.4 (2004).
- [5] Z. Yang, X. Gao, W. Zhou, S. Jiao, Y. Zhang, and X. Jin. “Deformable 3D Gaussians for High-Fidelity Monocular Dynamic Scene Reconstruction”. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2024.
- [6] M. Zwicker, H. Pfister, J. Van Baar, and M. Gross. “EWA Volume Splatting”. In: Visualization. IEEE. 2001.